



# Herencia

Herencia, clases concretas, clases abstractas, generalización y especialización, this y super

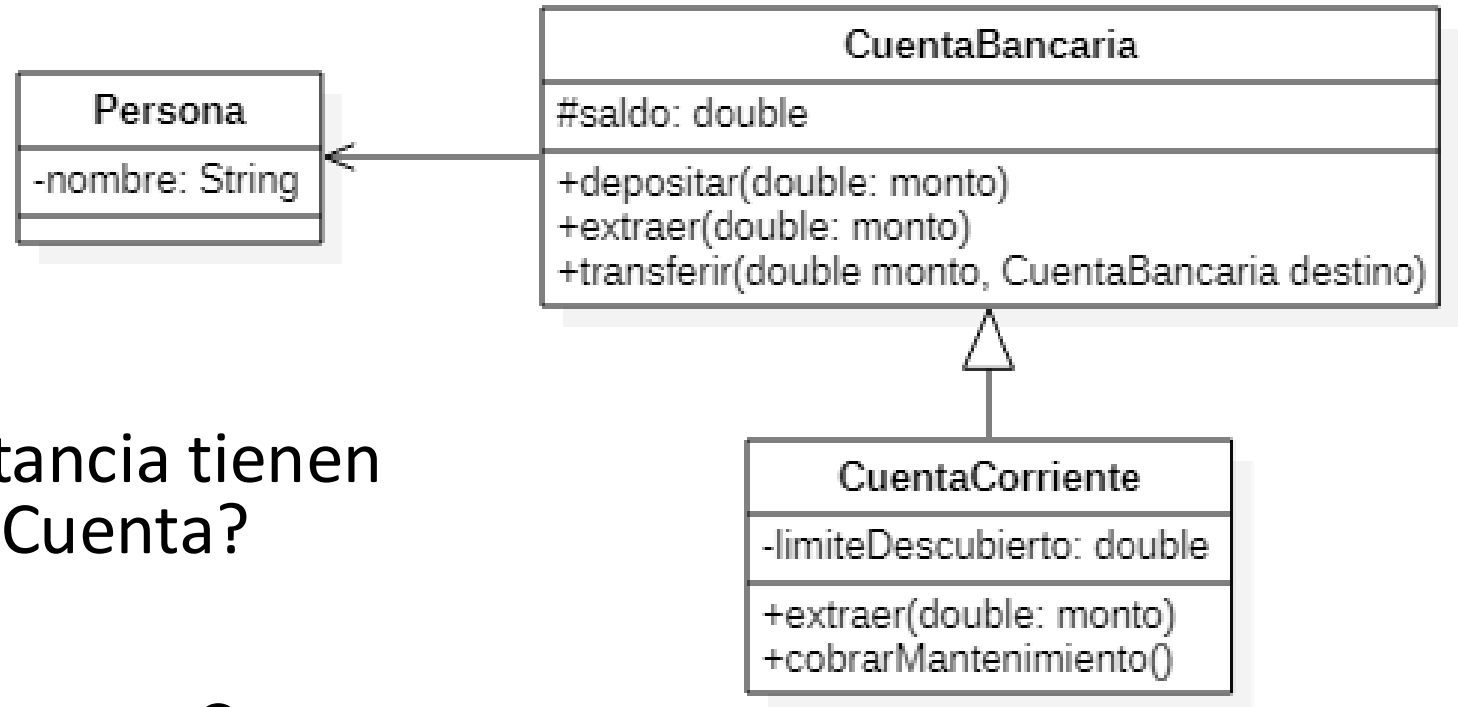
# Herencia

- Mecanismo que permite a una clase “heredar” estructura y comportamiento de otra clase
  - Es una estrategia de reúso de código
  - Es una estrategia de reúso de conceptos/definiciones
- En Java solo tenemos herencia simple
- Es una característica transitiva

```
public class CuentaCorriente extends CuentaBancaria {
```



# Herencia



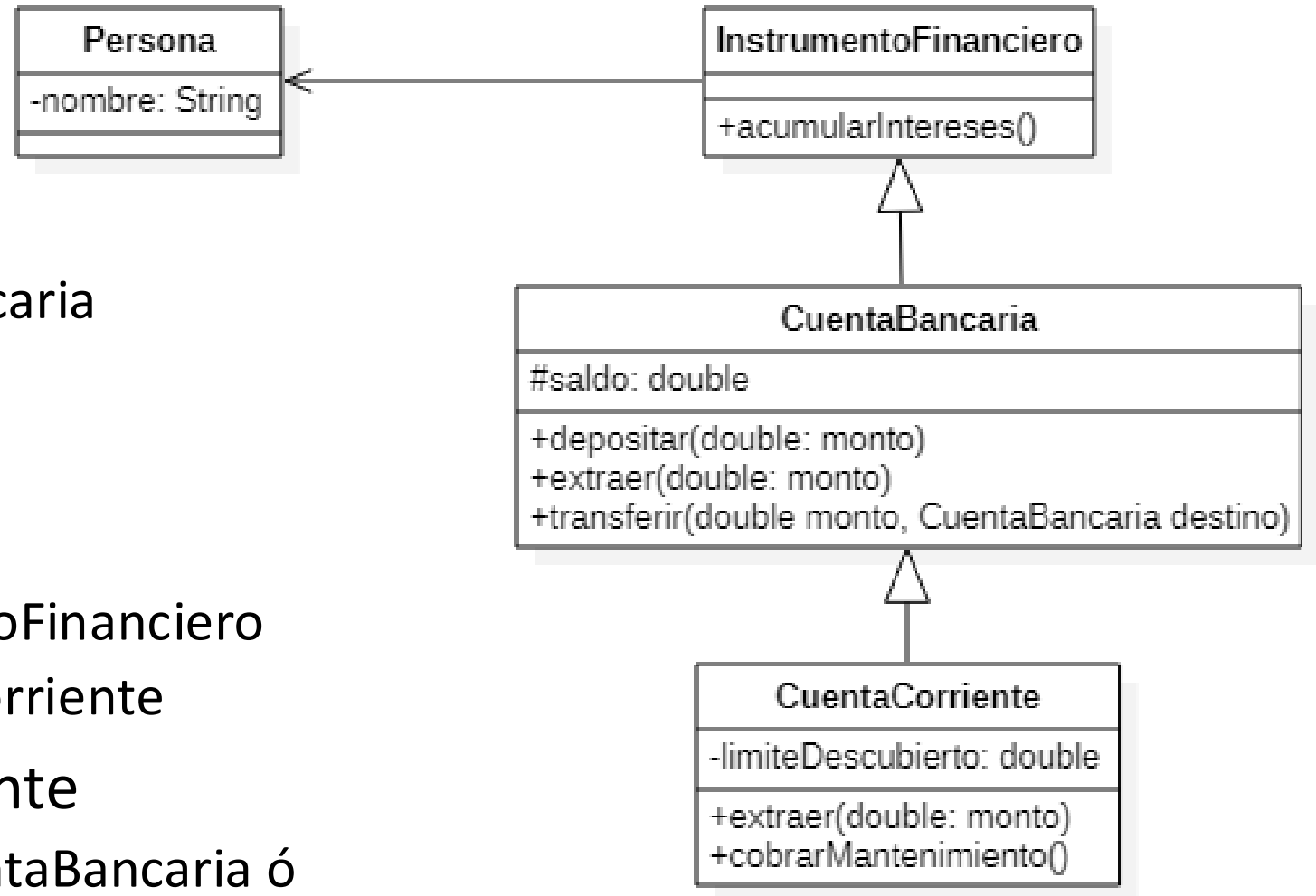
¿Qué variables de instancia tienen unaCuenta y otraCuenta?

¿Qué mensajes entienden unaCuenta y otraCuenta?

```
CuentaBancaria unaCuenta = new CuentaBancaria(alguien);
CuentaCorriente otraCuenta = new CuentaCorriente(alguien, 0, 1000);
```

# Vocabulario

- CuentaCorriente
  - es subclase de CuentaBancaria
  - hereda de CuentaBancaria
  - extiende CuentaBancaria
- CuentaBancaria
  - es subclase de InstrumentoFinanciero
  - es superclase de CuentaCorriente
- extraer() en CuentaCorriente
  - extiende extraer() de CuentaBancaria ó
  - redefine extraer() de CuentaBancaria

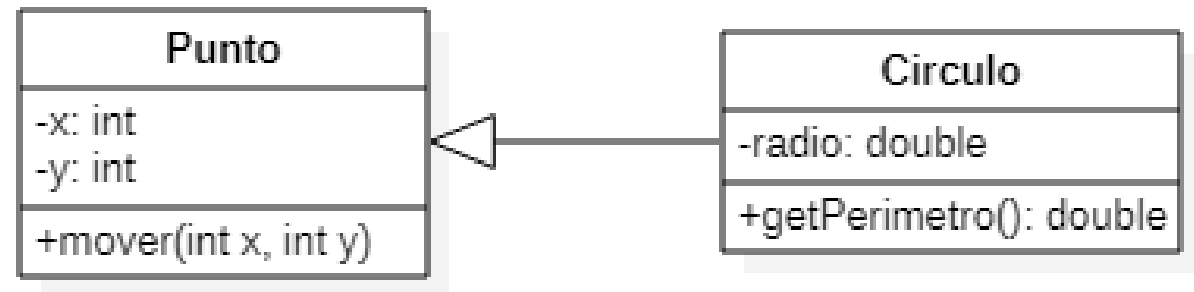


# La prueba “es un” ...

- Preguntarse “es-un” es la regla para identificar usos adecuados de herencia
  - Si suena bien en el lenguaje del dominio, es probable que sea un uso adecuado
  - Una cuenta corriente es una cuenta, una ventana de texto es una ventana, un array es una colección, un robot es un agente, ...
- Un anti-ejemplo famoso (herencia para construcción):



Un Círculo NO-ES-UN Punto,  
conoce o tiene un punto



# Principio de substitución de Liskov

- Si en algún lugar de nuestro programa estamos usando una clase, y esta clase es extendida, entonces tenemos que poder usar cualquiera de sus sub-clases y que el programa siga siendo válido (que pase los test).
- Nos ayuda a verificar que los criterios de subclasificación siguen funcionando.
- Es mucho más preciso que el intuitivo “es un”.

Barbara Liskov



# Method Lookup (recordamos)

- Cuando un objeto recibe un mensaje, se busca en su clase un método cuya firma se corresponda con el mensaje.
  - En un lenguaje dinámico, podría no encontrarlo (error en tiempo de ejecución)
  - En un lenguaje con tipado estático sabemos que lo entenderá (aunque no sabemos lo que hará)

| + Luz                    |
|--------------------------|
|                          |
| +encender()<br>+apagar() |

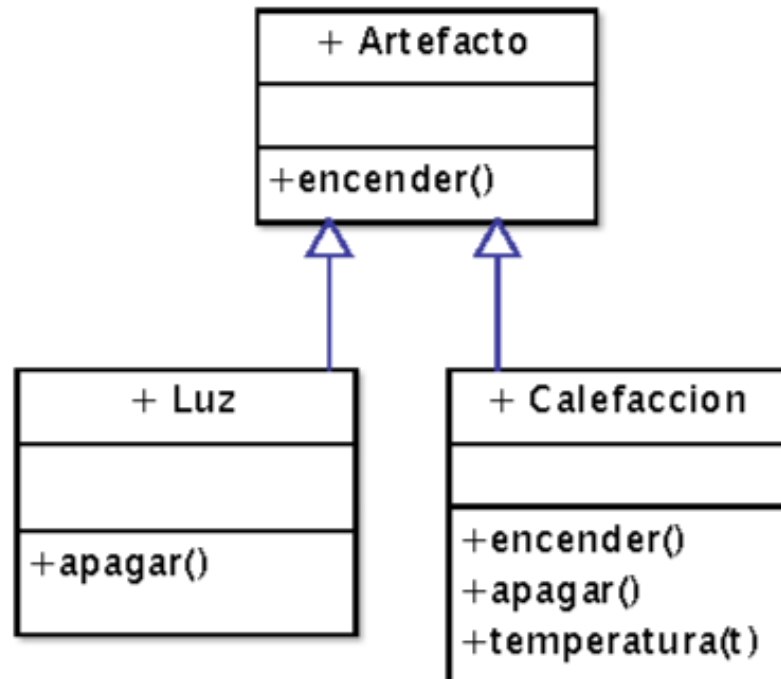
| + Calefaccion                               |
|---------------------------------------------|
|                                             |
| +encender()<br>+apagar()<br>+temperatura(t) |

| + Puerta              |
|-----------------------|
|                       |
| +abrir()<br>+cerrar() |

```
(new Luz()).encender();  
(new Calefaccion()).encender();  
(new Puerta()).encender();
```

# Method Lookup con herencia

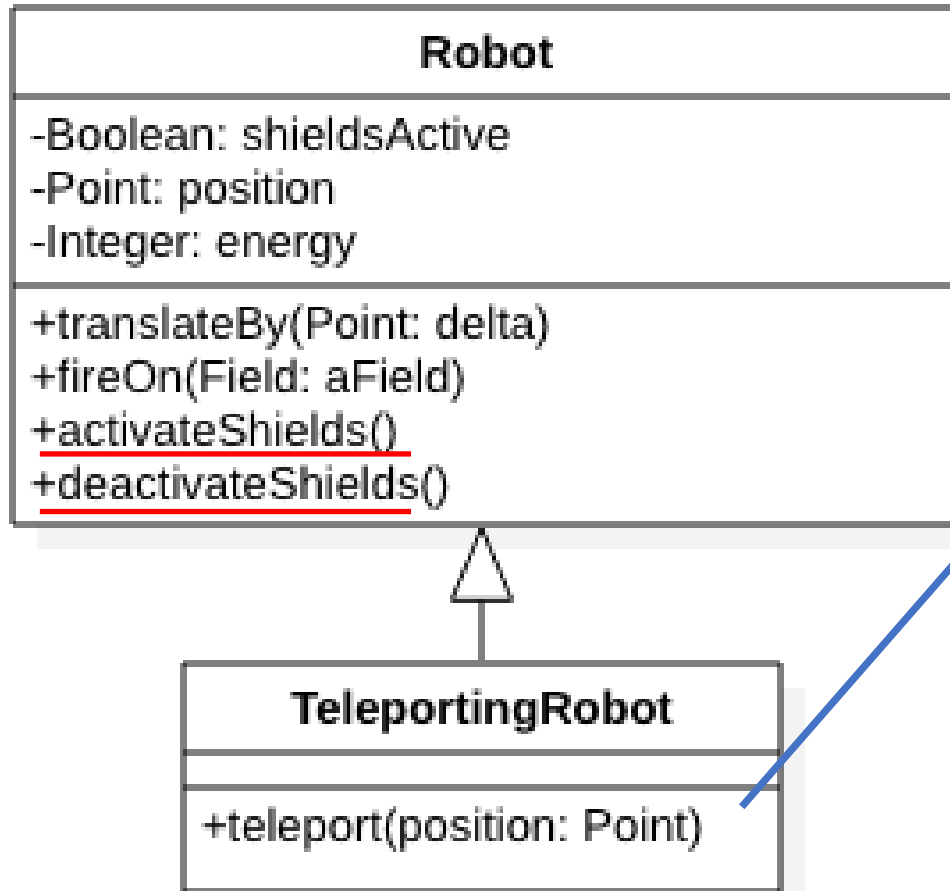
- Cuando un objeto recibe un mensaje, se busca en su clase un método cuya firma se corresponda con el mensaje. Si no lo encuentra, sigue buscando en la superclase de su clase, y en la superclase de esta ...



```
(new Luz()).encender();  
(new Calefaccion()).encender();
```



# Aprovechar comportamiento heredado

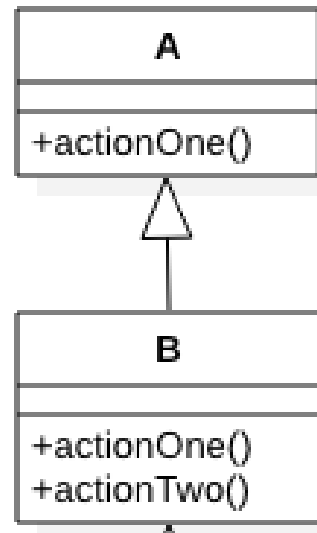


```
public void teleport(Point2D position) {  
    this.activateShields();  
    this.position = position;  
    this.deactivateShields();  
}
```

# Sobre escribir métodos (overriding)

- La búsqueda en la cadena de superclases termina tan pronto encuentro un método cuya firma coincide con la que busco.
- Si heredaba un método con la misma firma, el mismo queda “oculto” (redefinir / override)
- No es algo que ocurra con frecuencia (puede dar mal olor)

```
(new B()).actionOne();
```



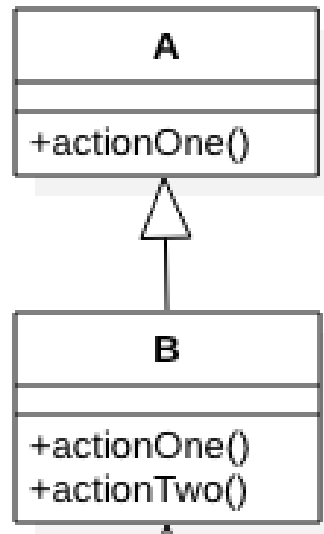
```
public void actionOne() {  
    // Hacer algo como le gusta a A  
}
```

```
public void actionOne() {  
    // Hacer algo como le gusta a B  
}
```

# Extender métodos

- ¿Qué hacemos si queremos aprovechar (extender) ese método que heredábamos?

```
(new B()).actionOne();
```



```
public void actionOne() {  
    // Hacer algo como le gusta a A  
}
```

```
public void actionOne() {  
    // Hacer algo como le gusta a B  
    // Hacer algo como le gusta a A  
    // Hacer algo como le gusta a B  
}
```

# super

- **Podemos pensar a super** como una “pseudo-variable” (como this)
  - no puedo asignarle valor
  - Toma valor automáticamente cuando un objeto comienza a ejecutar un método
- En un método, “**super y this**” **hacen** referencia al objeto que lo ejecuta (al receptor del mensaje)
- Utilizar super en lugar de this “solo cambia la forma en la que se hace el method lookup”
- Se utiliza para:
  - Solamente para extender comportamiento heredado (reimplementar un método e incluir el comportamiento que se heredaba para él).

# super y el method lookup

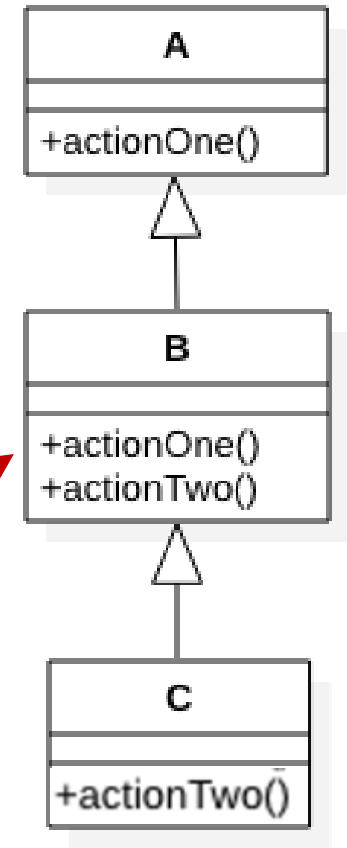
Cuando **super** recibe un mensaje, la búsqueda de métodos comienza en la clase **inmediata superior** a aquella donde está definido el método que envía el mensaje (sin importar la clase del receptor)

```
(new A()).actionOne();
```

```
(new B()).actionOne();
```

```
(new C()).actionOne();
```

```
public void actionOne() {  
    this.actionTwo();  
    super.actionOne();  
    this.actionTwo();  
}
```



# Super() en los constructores

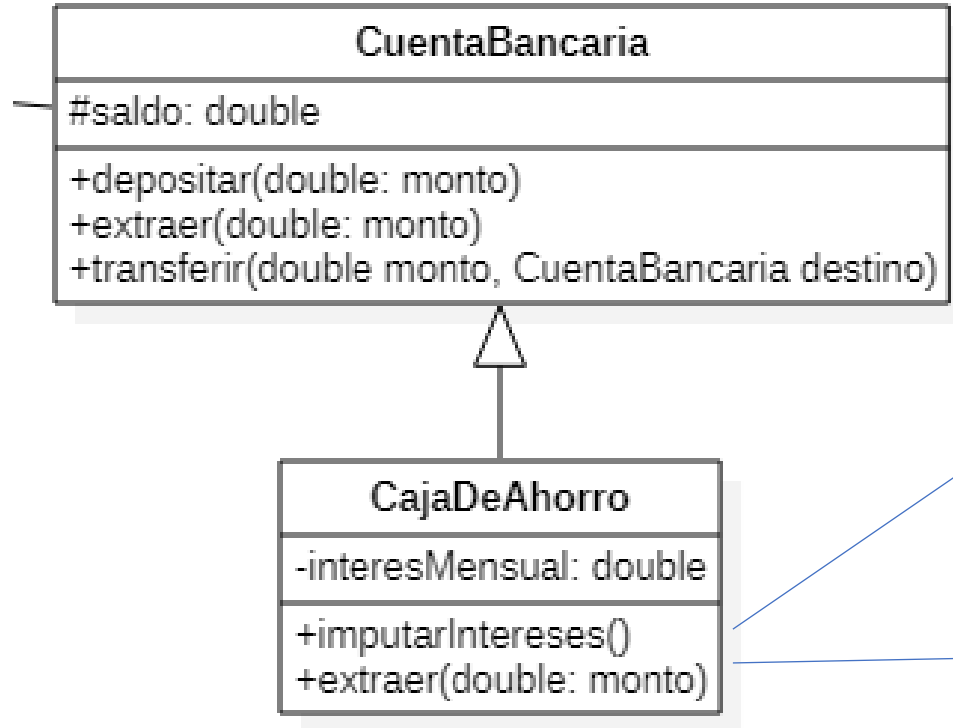
- Los constructores en Java son subrutinas que se ejecutan en la creación de objetos - no se heredan
- Si quiero reutilizar comportamiento de otro constructor debo invocarlo explícitamente - usando `super(...)` al principio

```
public CuentaBancaria(Persona titular) {  
    this.titular = titular;  
}
```

```
public CuentaCorriente(Persona titular, double saldoInicial, double limiteDescubierto) {  
    super(titular);  
    saldo = saldoInicial;  
    this.limiteDescubierto = limiteDescubierto;  
}
```

# Especializar

- Especializar: crear una subclase especializando una clase existente



```
public void extraer(double monto) {  
    saldo = saldo - monto;  
}
```

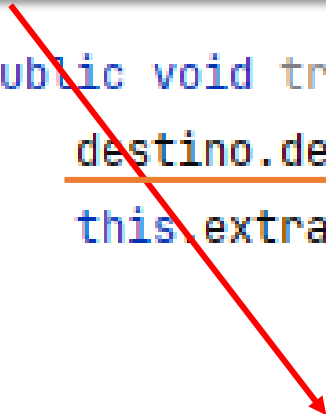
```
public void imputarIntereses() {  
    this.depositar(interésMensual * saldo);  
}
```

```
public void extraer(double monto) {  
    if (monto <= saldo) {  
        super.extraer(monto);  
    }  
}
```

# Clase abstracta

- Una clase abstracta captura comportamiento y estructura que será común a otras clases
- Una clase abstracta no puede tener instancias (no es completo)
- Seguramente será especializada
- Puede declarar comportamiento abstracto y utilizarlo para implementar comportamiento concreto

**Gancho que deben implementar las subclases**



```
public abstract class CuentaBancaria {  
    protected double saldo;  
  
    public void depositar(double monto) {  
        saldo = saldo + monto;  
    }  
  
    public void transferir(double monto, CuentaBancaria  
        destino) {  
        destino.depositar(monto);  
        this.extraer(monto);  
    }  
  
    public abstract void extraer(double monto);  
}
```



# Generalizar

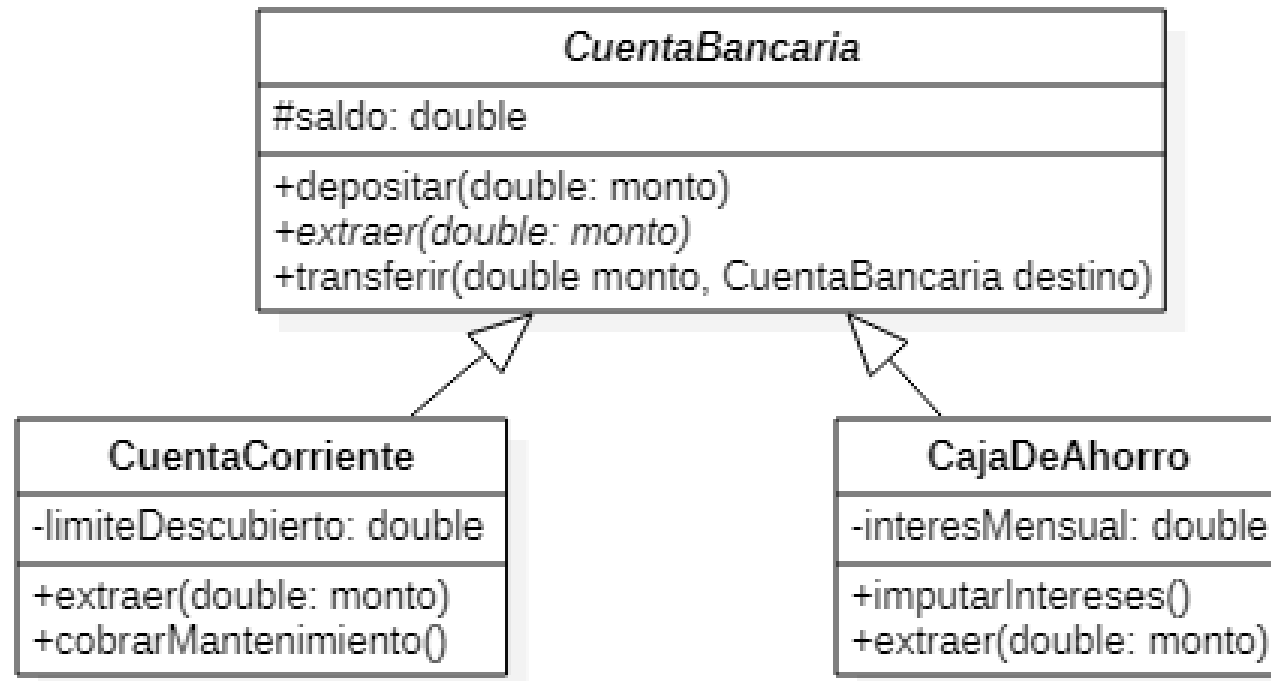
- Generalizar: Introducir una superclase que abstrae aspectos comunes a otras – suele resultar en una clase abstracta

| CuentaCorriente                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------|
| -limiteDescubierto: double<br>-saldo: double                                                                                         |
| +depositar(double: monto)<br>+transferir(double monto, CuentaCorriente destino)<br>+extraer(double: monto)<br>+cobrarMantenimiento() |

| CajaDeAhorro                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------|
| -interesMensual: double<br>-saldo: double                                                                                      |
| +depositar(double: monto)<br>+transferir(double monto, CajaDeAhorro destino)<br>+extraer(double: monto)<br>+imputarIntereses() |

# Generalizar

- Generalizar: Introducir una superclase que abstraee aspectos comunes a otras – suele resultar en una clase abstracta



# Situaciones de uso de herencia

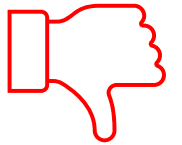
- Deseadas

- Subclasificar para especializar
- Herencia para especificar
- Subclasificar para extender

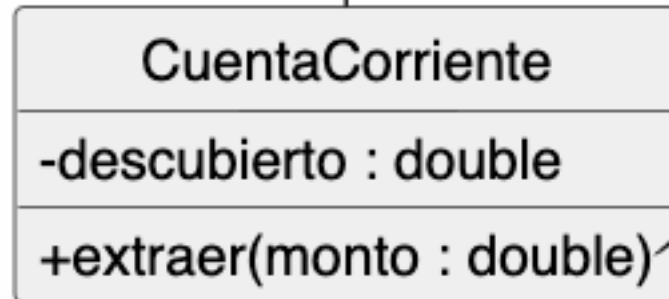
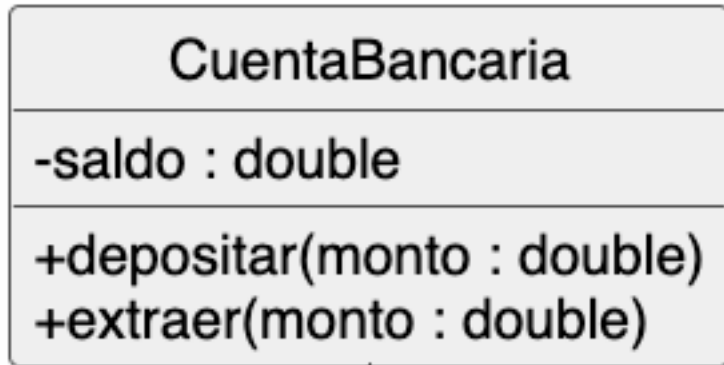


- No deseadas

- Heredar para construir
- Subclasificar para generalizar (no es lo mismo que generalización)
- Subclasificar para limitar
- Herencia indecisa (subclasificación por varianza)



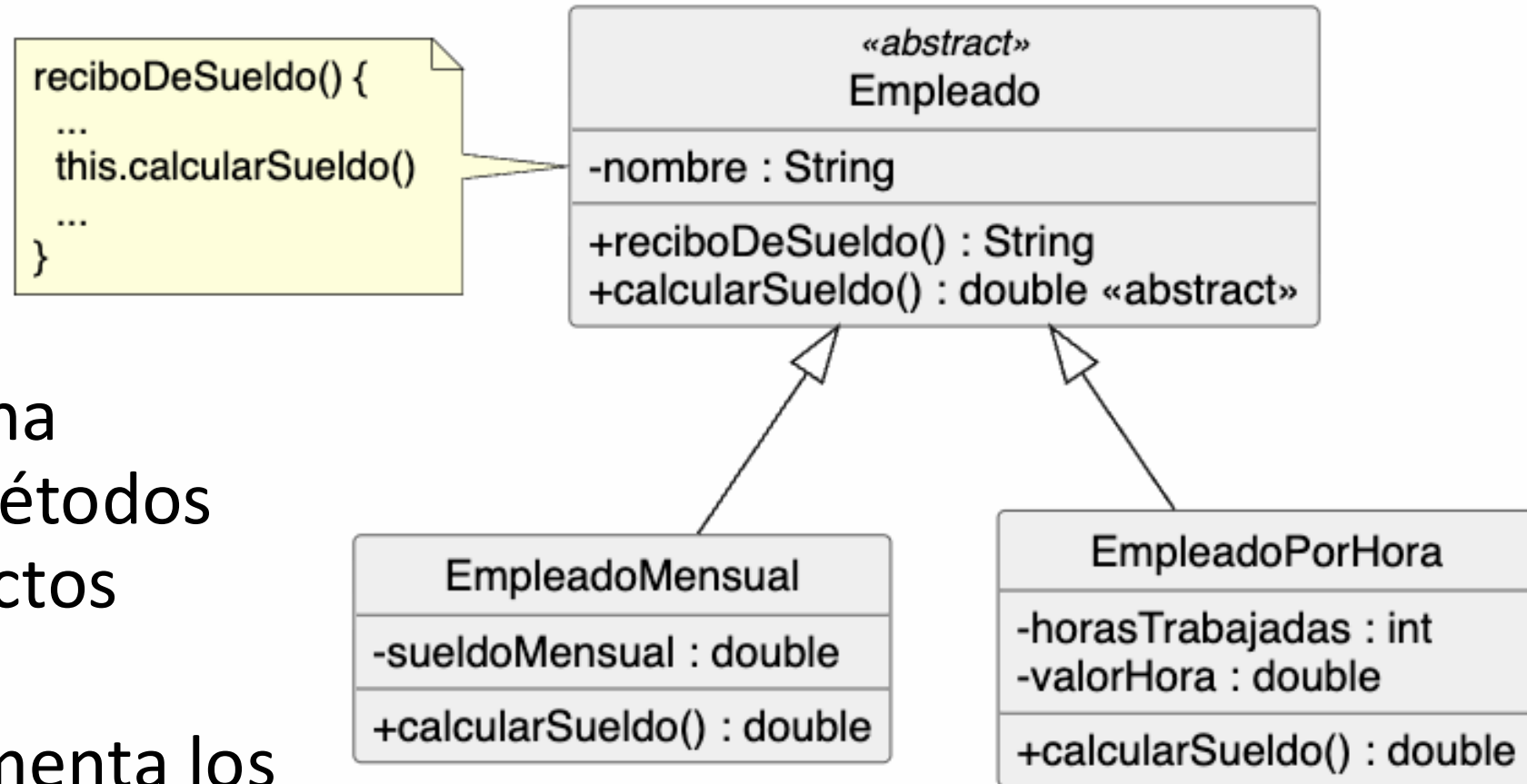
# Subclasificar para especializar



especializa extraer()  
usando super.extraer(...)

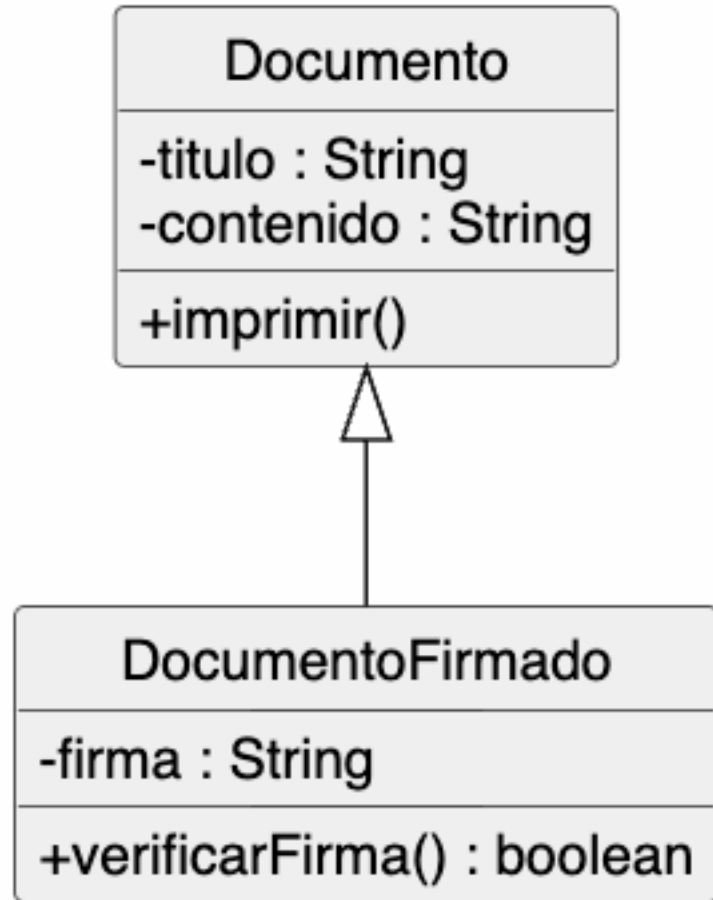
- La subclase extiende métodos método para especializarlos
- Ambas clases concretas

# Herencia para especificar



- La superclase es una combinación de métodos concretos y abstractos (clase abstracta)
- La subclase implementa los métodos abstractos

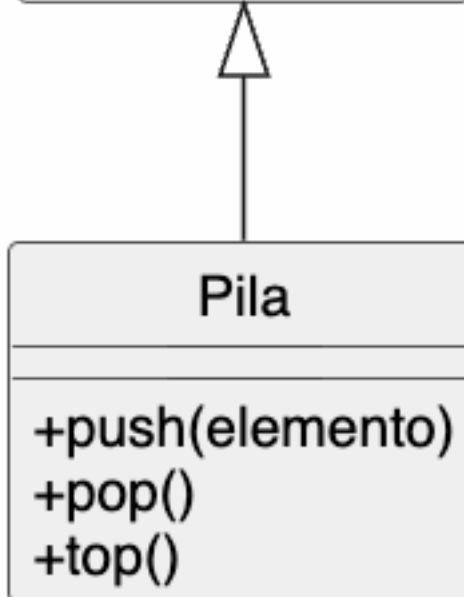
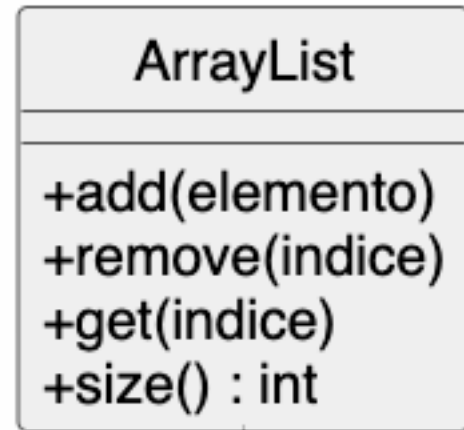
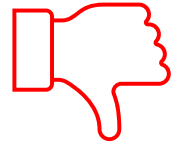
# Subclasificar para extender



- La subclase agrega nuevos métodos
- Ambas son concretas (se puede dar también siendo abstracta la superclase)

Agrega comportamiento:  
`verificarFirma()`

# Heredar para construir

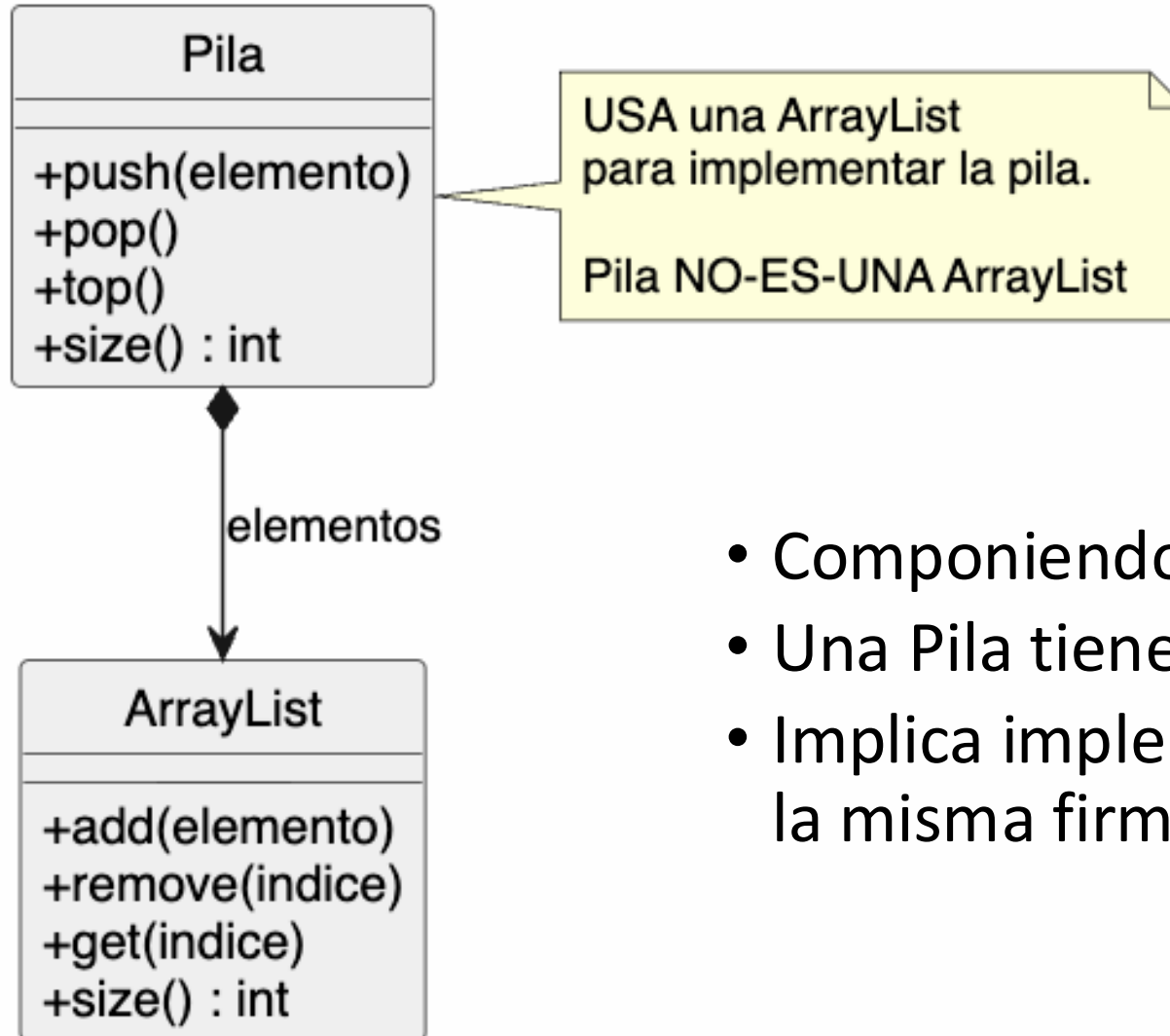


- Heredo comportamiento y estructura, pero cambio el tipo (no-es-un)
  - En una pila no quiero remove, get y add
- Se debe evitar, aunque nos vamos a cruzar con ejemplos (en código de otros)
- Si es posible, reemplazar por composición

Hereda estructura y comportamiento para reutilizar la implementación.

Pero:  
Pila NO-ES-UNA ArrayList

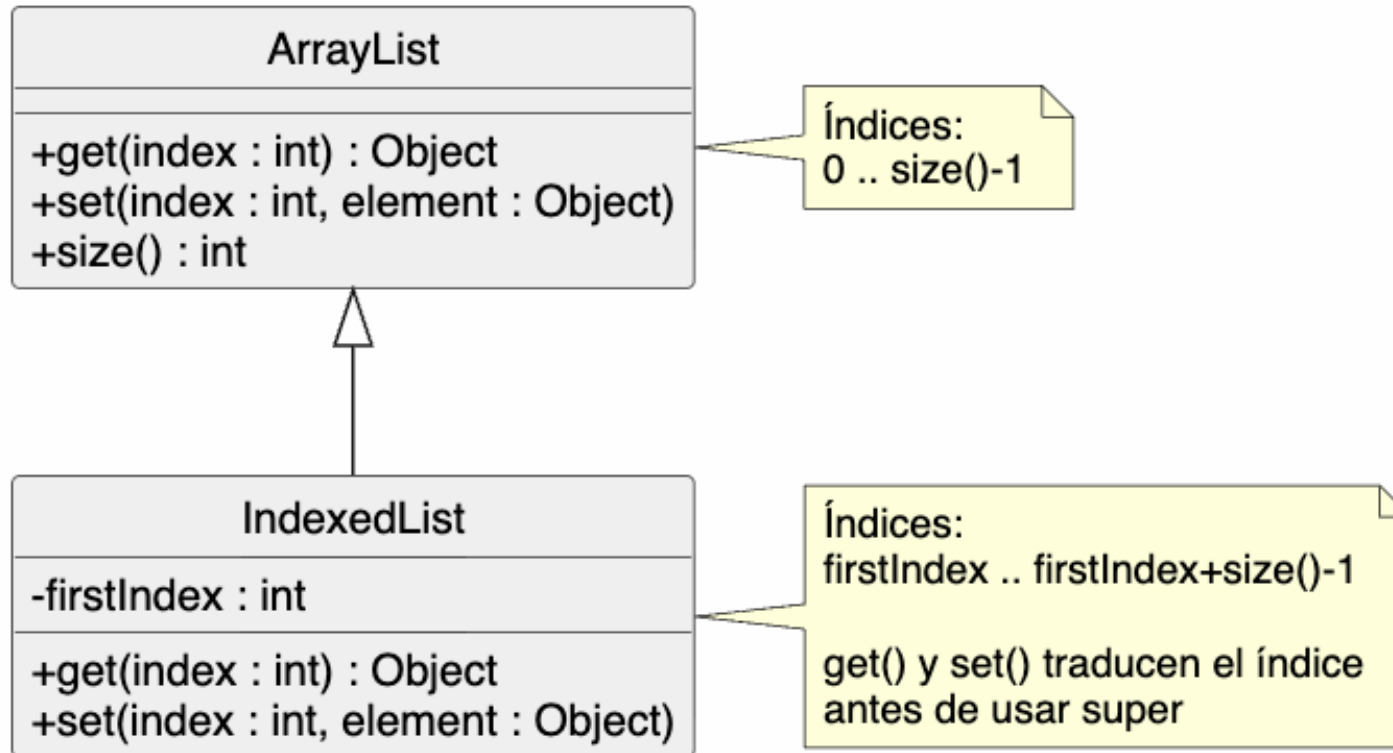
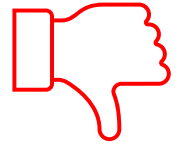
# Componer para construir



- Componiendo, ese mismo caso seria así
- Una Pila tiene un ArrayList (es una parte suya)
- Implica implementar incluso métodos que tienen la misma firma que en ArrayList (delegando)

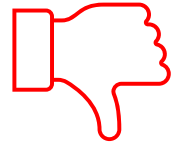


# Subclasificar para generalizar

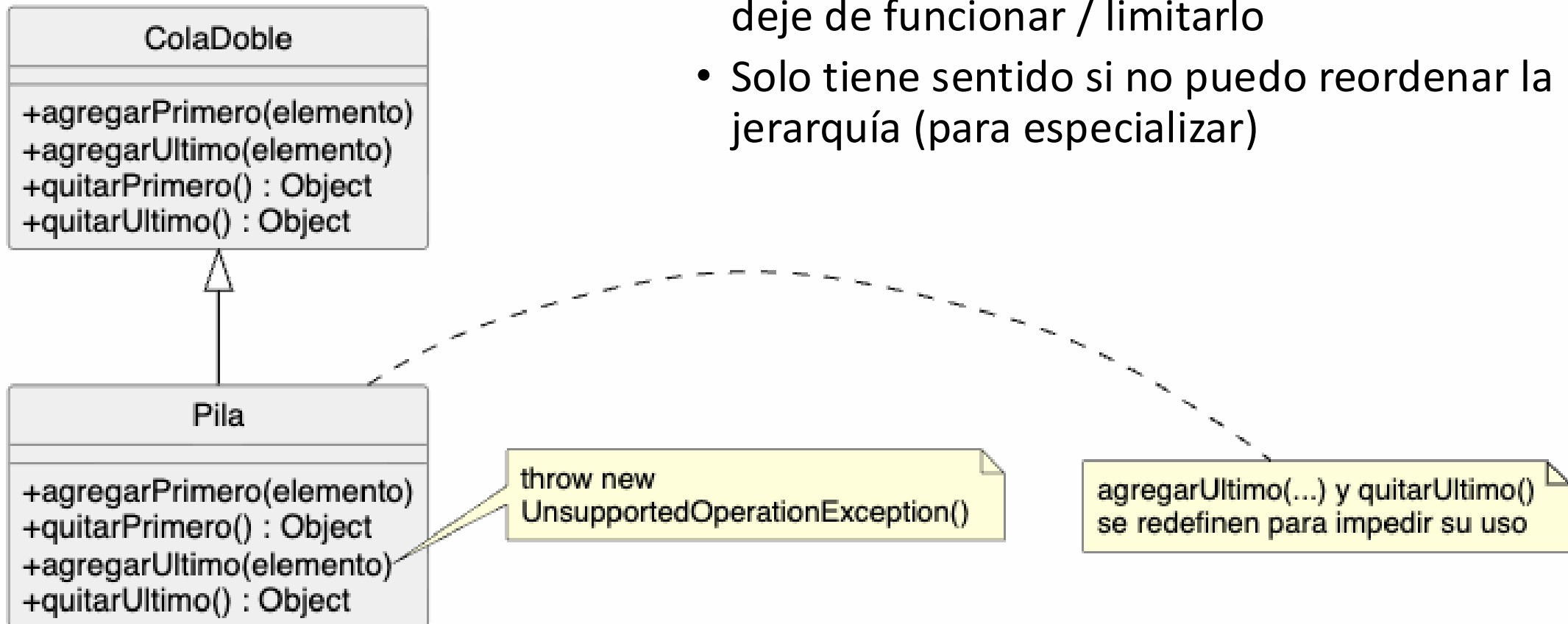


- La subclase funcionalidad métodos para hacerla más generales
- Solo tiene sentido si no puedo reordenar la jerarquía (para especializar)
- No confundir con generalización

# Subclasificar para limitar



- La subclase reimplementa un método para que deje de funcionar / limitarlo
- Solo tiene sentido si no puedo reordenar la jerarquía (para especializar)



# Tipos/tipado en lenguajes OO (recordamos)

- Tipo = Conjunto de firmas de operaciones/métodos (nombre, tipos de argumentos)
- Con eso en mente:
  - Cada clase en Java define un tipo
  - Cada interfaz en Java define un tipo
  - Cuando una clase implementa una interfaz, es un **sub-tipo** de ella
  - Cuando una clase subclasifica a otra, es un **sub-tipo** de ella
- El tipo de una variable, es el que indico al declararla
- El tipo de un método, es el tipo de lo que retorna
- El tipo de un constructor, es la clase que construye

# Tipos, variables, instancias, mensajes

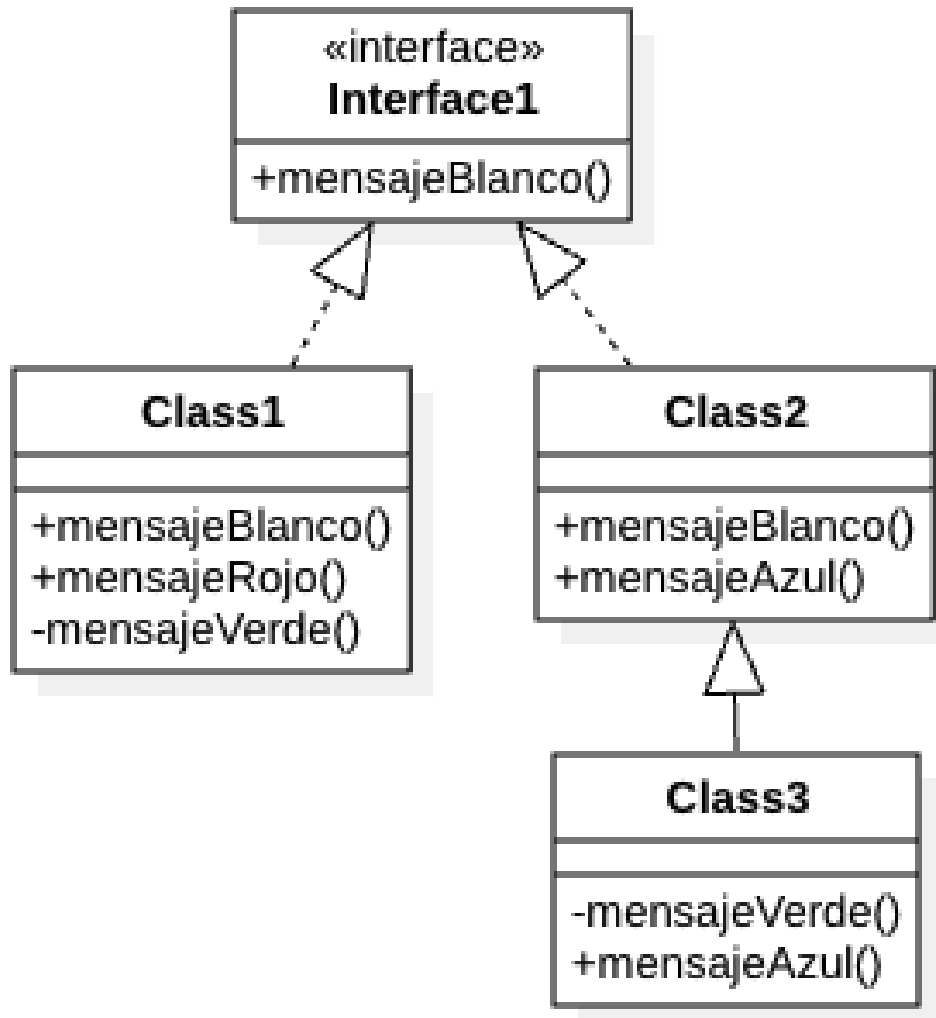
## En compilación

- Cada variable tiene un tipo
- En cada asignación se chequea que el tipo del valor (derecha) sea el mismo o un subtipo del de la variable (izquierda)
- En cada envío de mensaje se chequea que el tipo de la variable incluya la firma del mensaje

## En ejecución

- Cada objeto es instancia de una clase (eso nunca cambia)
- Cuando el objeto recibe un mensaje, el método se comienza a buscar en su clase (binding dinámico)
- El tipo de la variable que apunta al objeto “no importa”

# Compilando ...



```
Interface1 obj1 = new [redacted]
obj1.[redacted]();
```

```
[redacted] obj3 = new Class3();
obj3.mensajeAzul();
```

```
List[redacted] lista = new ArrayList[redacted]();
//...
for ([redacted] elemento : lista) {
    elemento.mensajeBlanco();
}
```

```
[redacted] obj4 = new [redacted]();
obj4.mensajeVerde();
```

# Clases abstractas e interfaces

- Una clase abstracta “es una clase”
  - Puedo usarla como tipo
  - Puede o no tener métodos abstractos
  - Ofrece implementación a algunos métodos
  - Sus métodos concretos pueden depender de sus métodos abstractos (p.e., si se define antes de sus subclases)
- Una interfaz define un tipo
  - Sirve como contrato
  - Puede extender a otras
- Se pueden implementar muchas interfaces pero solo se puede heredar de una clase

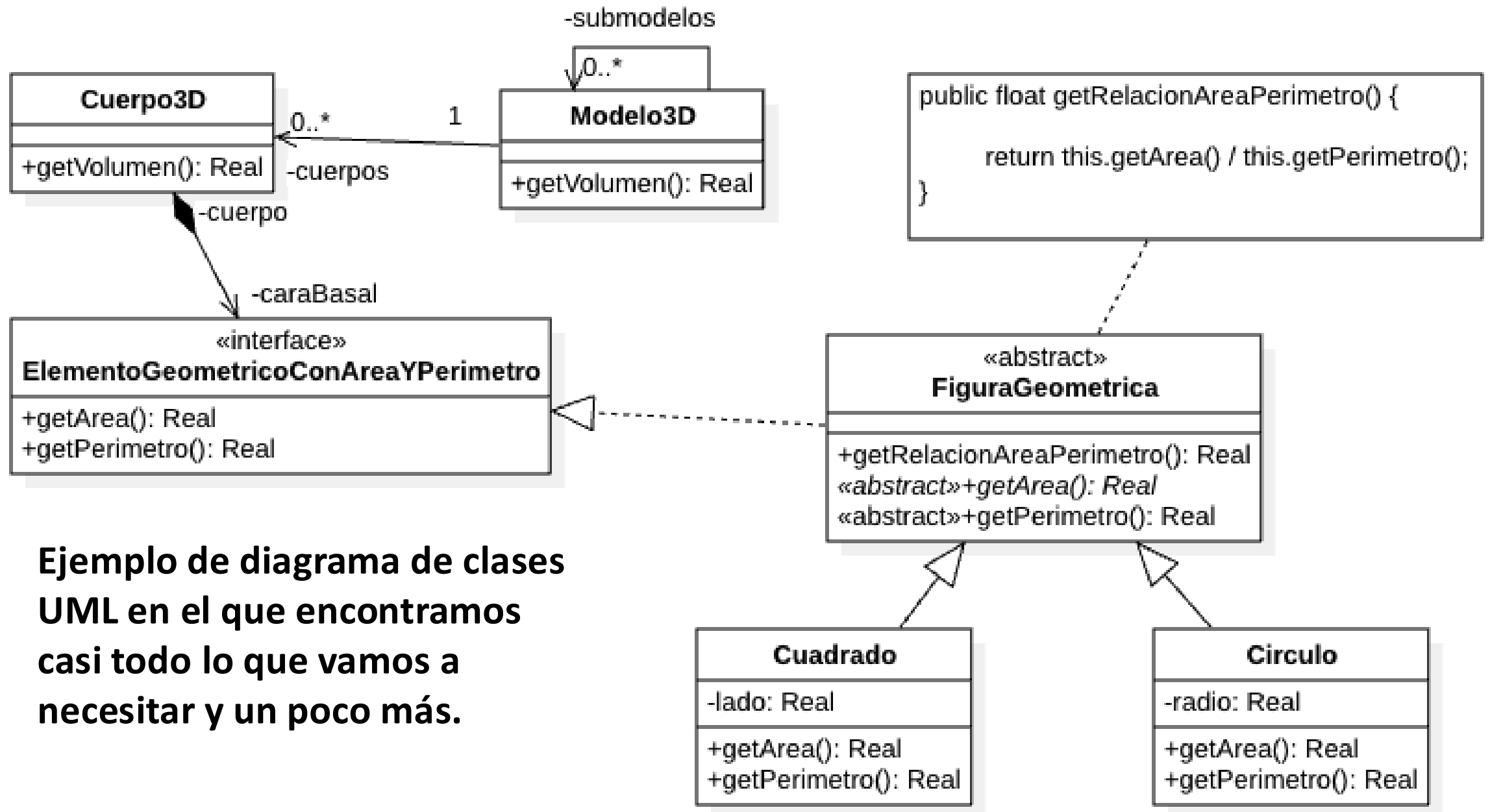
# Modificadores de visibilidad con herencia

- Si declaro una variable de instancia como `private` en una clase A:
  - Las instancias de las subclases de A tendrán esa variable de instancia
  - En los métodos de las subclases de A, no puedo hacer referencia ella
- Si declaro un método “`m()`” como `private` en una clase A:
  - Las instancias de sus subclases entenderán el mensaje `m()`
  - En los métodos de las subclases de A no puedo enviar `m()` a “`this`”
- Si declaro variables y métodos como `protected`, puedo verlos en las subclases
- ¿Qué impacto tiene restringir la visibilidad de esta manera en términos de acoplamiento?

# Visibilidad, herencia y OO1

- El ocultamiento de información favorece el bajo acoplamiento
  - Por eso en algunos contextos se mantienen las variables de instancia como privadas incluso en el contexto de herencia
- En OO1 vamos a utilizar la siguiente estrategia:
  - Las variables de instancia son siempre privadas.
  - Si en una subclase, para extender o especializar, dependo de ese conocimiento, cambio la visibilidad de mis variables a **protected (# en los diagramas de UML)**.
  - Debo saber (y actuar con cuidado) que eso las hace públicas a otras clases del mismo paquete (incluso si no son subclases)





**Ejemplo de diagrama de clases UML en el que encontramos casi todo lo que vamos a necesitar y un poco más.**

# Lectura recomendada

*An Introduction to Object-Oriented Programming,*

Timothy Budd

- Capitulo 8

